

Question 1

Introduction

The objective of this task is to analyze the UCI Energy Efficiency dataset and build a regression pipeline to predict two main targets, Heating Load as in Y1, and Cooling Load as in Y2. Building efficiency is a topic that is critical in modern construction when it can impact long term costs and sustainability. The analysis that I now have performed focuses on understanding the data's behavior, justifying my preprocessing steps. And exploring why one target appears harder to one model than the other.

Dataset Understanding

This dataset consists of 768 samples with 8 input features and 2 targets. While doing the initial check I found no missing values. I found that X6 (orientation) and X8 (Glazing Area Distribution) are categorical variables encoded at integers, while all the other features were stored as numbers. Since the data comes from a simulation study, the feature values represent fixed design configurations rather than random real-world measurements.

The eight input features are (*Tsanas, 2012, Energy*):

X1	Relative Compactness
X2	Surface Area
X3	Wall Area
X4	Roof Area
X5	Overall Height
X6	Orientation

X7	Glazing Area
X8	Glazing Area Distribution
Y1	Heating Load
Y2	Cooling Load

The dataset may not reflect either variability or the noise present in real-world energy measurements because the dataset is synthetically generated through building simulations. The values fixed for

features like for example X6 mean that the dataset won't capture gradual transitions between design configurations. The consequence of this is that it will limit the generalizability of any trained model to real-world buildings.

Methodology

No missing values were found, that's why I focused on scaling. I standardized all features using StandardScaler before training for linear regression. I used Random Forest and applied the same scaling to both models to ensure the best comparison possible. I kept X6 and X8 as integers because their discrete nature works really well with tree-based models. I also decided that one-hot encoding would probably add unnecessary dimensionality without that much performance gain.

I calculated a pearson correlation matrix to identify linear relationships between variables.



This correlation matrix shows that X5 has the strongest correlation with Y1 and Y2. While X6 shows near-zero correlation with both targets. I observed that there's multicollinearity between X1 and X2 ($r=0.99$), and between X4 and X5 ($r=0.97$).

X5 still stands out as the strongest predictor. The extreme multicollinearity between X1,X2 and X4,X5 is bad for linear models, as it can make coefficient estimates unstable. One thing I found interesting is that Y1 and Y2 are almost perfectly correlated, which means that the design factors driving heating needs are nearly identical to those driving cooling.

I chose two models whereas Linear Regression as a simple interpretable baseline, and Random Forest as a non-linear model. I chose Random Forest because of the discrete structure of the data, as trees are good at capturing these types of differences.

A multi-output regression was also evaluated because Y1 and Y2 were highly correlated. The results are practically identical since Scikitlearn's MultiOutputRegressor essentially trains independent models for each target.

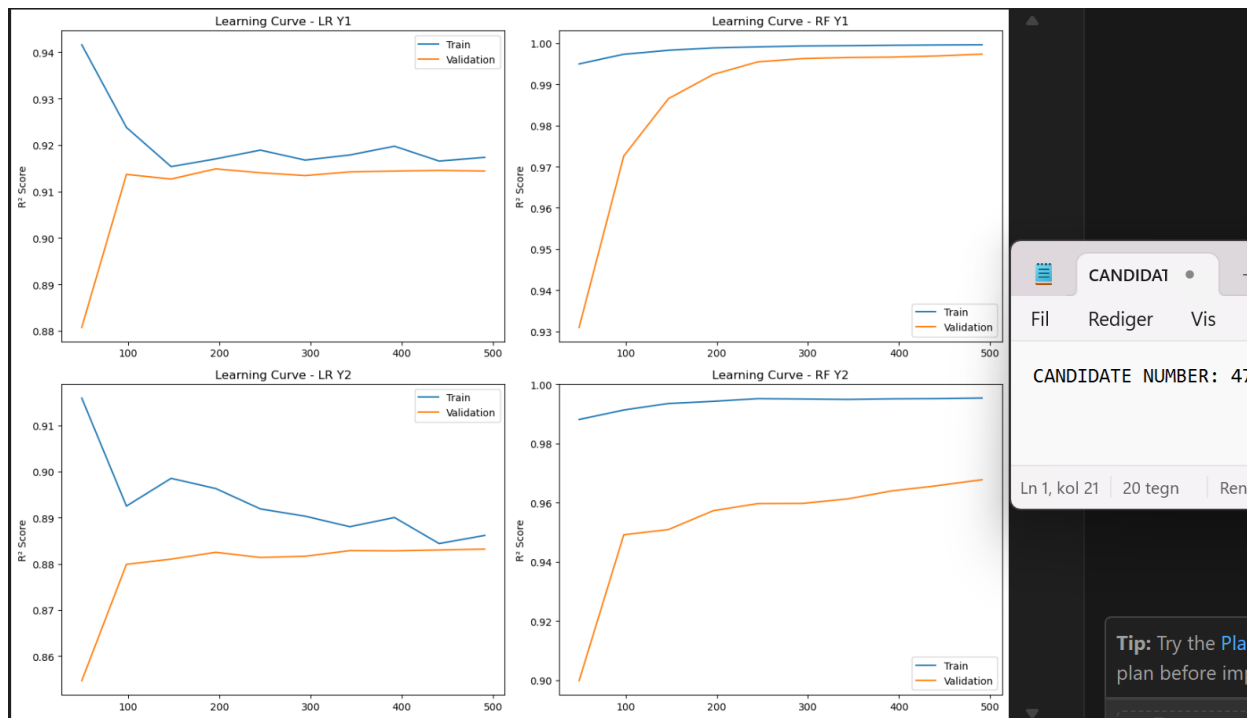
Results

This table sums up the performance of each model on the held-out set.

Y1 (Heating Load)	MAE	RMSE	R^2
Linear Regression	2.182	3.025	0.912
Random Forest	0.352	0.494	0.998
Y2 (Cooling Load)			
Linear Regression	2.195	3.145	0.893
Random Forest	1.071	1.741	0.967

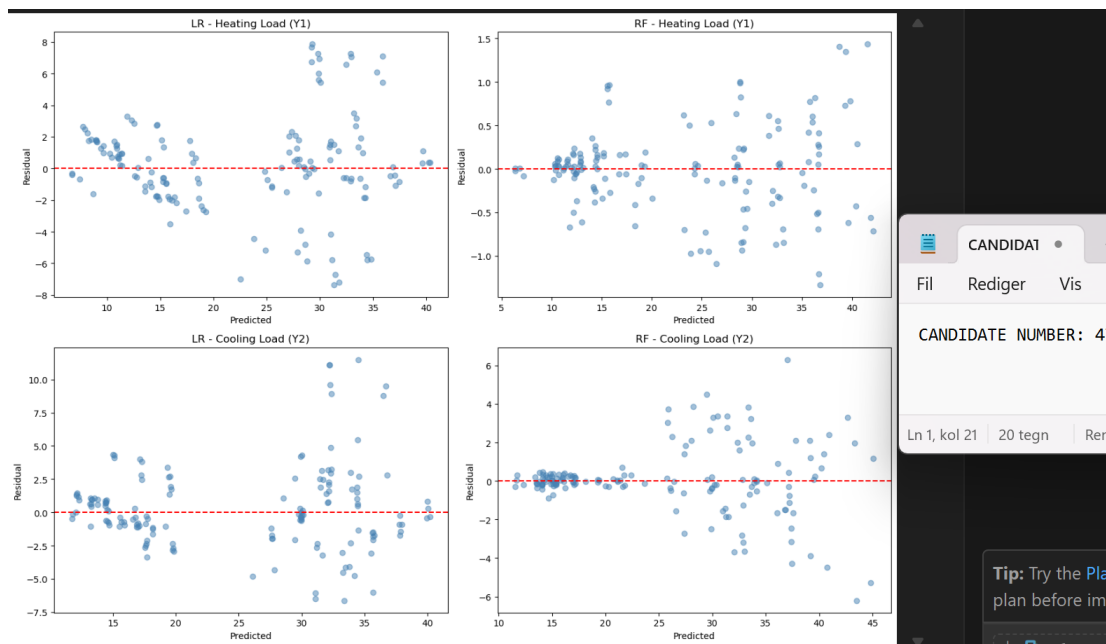
Regression model performance on the test. MAE = Mean Absolute Error (absolute differences between predicted and actual values), RMSE = Root Mean Squared Error (penalizes larger errors more heavily), R^2 = Coefficient Of Determination (where 1 is perfect).

Random forest outperformed Linear Regression when it achieved almost perfect scores ($R^2 = 0.998$). For Y2 the MAE is three times higher than for heating which confirms that cooling is a more difficult target for the model.



Learning curves showing train and validation r^2 scores. Linear regression shows high bias while Random Forest for Y2 shows a persistent gap that indicates mild overfitting.

The curves for linear regression show that the model cannot capture the underlying patterns. While Random Forest Y1 shows good convergence. However, Y2 shows a persistent gap between training and validation scores.



Residual plots (predicted vs residual). Linear regression shows systematic patterns (non-linearity). Random Forest for Y2 shows larger and more spread residuals than Y1.

Linear regression is the wrong tool for this data because it reveals non-linear patterns. Random Forest Y1 residuals are tiny and random while Y2 residuals are larger and more dispersed that matches the lower R^2 score.

Personal Reflection

If we look at the evidence from the histograms to the residual plots, I will say that Cooling Load (Y2) is harder to predict than Heating Load (Y1).

One thing that genuinely surprised me was how little X6 mattered. In theory a building's orientation should impact solar gain and energy load. Still the data showed almost zero correlation. This may be a byproduct of how the simulation was designed. If I could decide, I would consider removing redundant features like X1 or X2 to simplify the model and I would probably prioritize testing against actual measured data to see if those simulation born patterns hold up under noisy conditions.

Discussion

The Random Forest model is a good strength here because of its ability to handle the non-linearities of the dataset. While one weakness is that it was trained on clean data. The multicollinearity between features is also a risk when it can lead to difficulties when determining which of the features are truly driving the predictions.

If this was going to be deployed, the model would need to be retrained on real world meter data. Its worth noticing that since X6 showed no predictive value, its inclusion in a production level tool should be analyzed to avoid unnecessary data collection.

It is a risk to use a model like this for building certification or energy audits because if the training data is biased toward specific building types, the model could produce incorrect assessments for others. Decisions should be made with human experts. These predictions are tools we should be in control of, not something we should blindly rely on.

Question 2

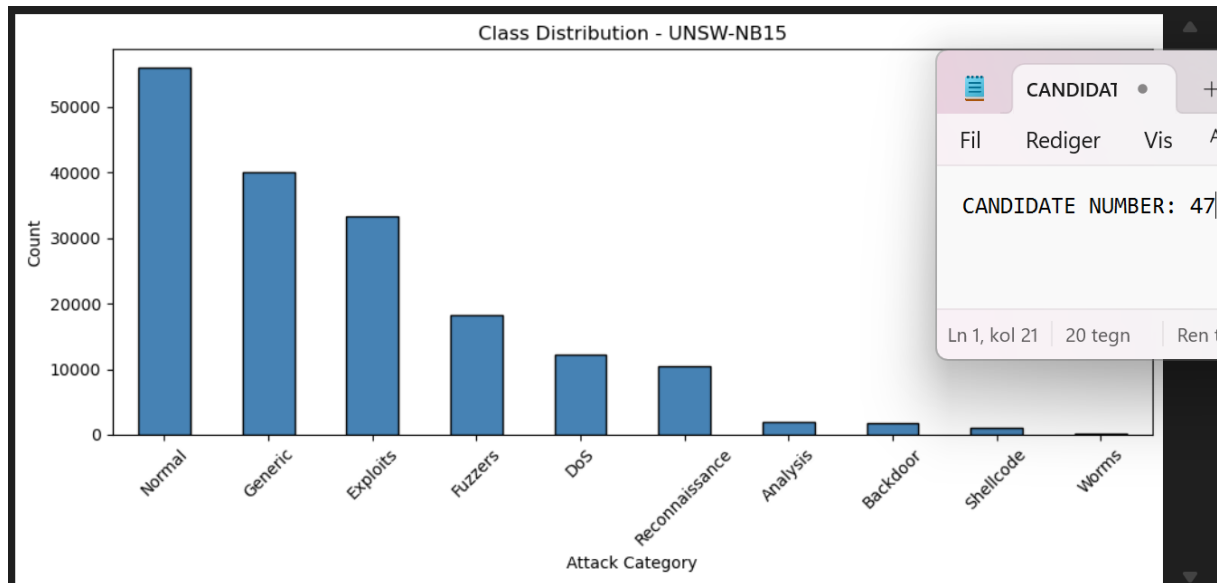
Introduction

Network intrusion detection plays a critical role in modern cybersecurity infrastructure. The task uses UNSW-NB15 dataset to build a multi class classification that distinguishes between normal traffic and

nine attack categories (*moustafa, 2015, UNSW-NB15*). The core problem is severe class imbalance, where minority attack types are underrepresented.

Dataset Understanding

This training set contains 175 341 samples and 34 features after preprocessing. Target variable `attack_cat` has 10 classes.



This shows a big imbalance where normal traffic accounts has 56 000 samples, while worms only have 130. This means that a naïve model could achieve high accuracy by predicting majority classes.

This dataset mainly contains three categorical features. Proto (133), service (13) and state (9). All other features are numerical.

Methodology

I applied Label Encoding to proto, service and state. I did not apply One Hot encoding because of its high cardinality (133 unique values). That would just introduce excessive dimensionality. Label Encoding would be a great choice here because Random Forest does not assume ordinality in encoded values.

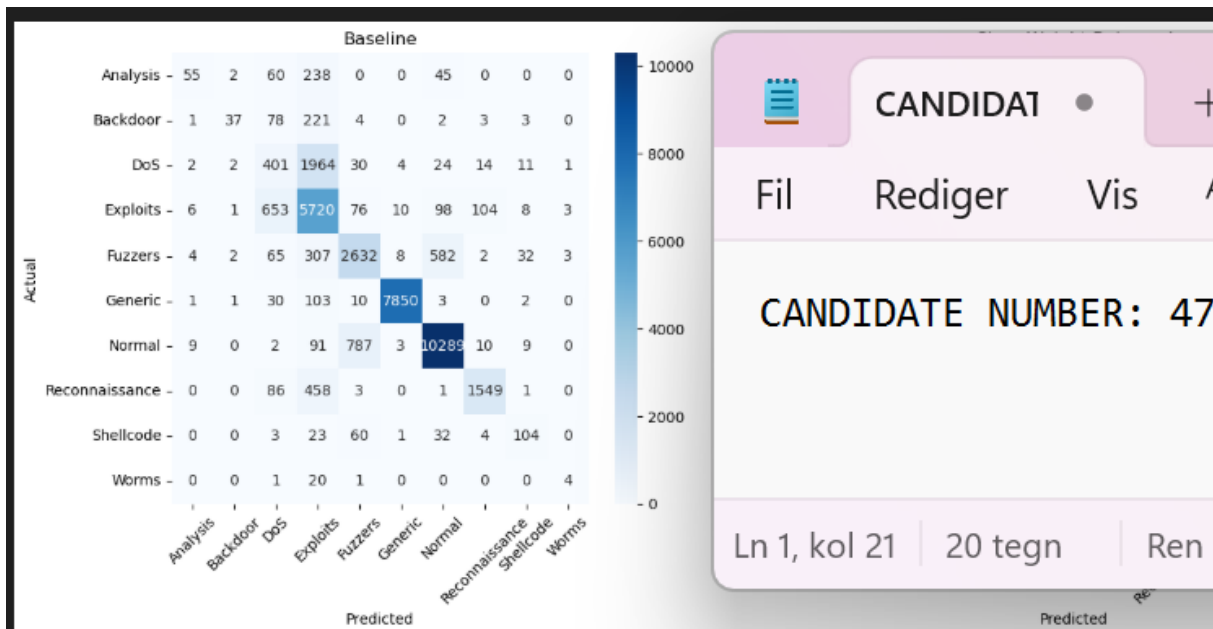
Random Forest was chosen as a method that reduces variance through bagging, as covered in the course material on ensemble learning. I could use Logistic Regression, but I chose Random Forest because it handles non-linear relationships and mixed feature types without extensive preprocessing.

Results

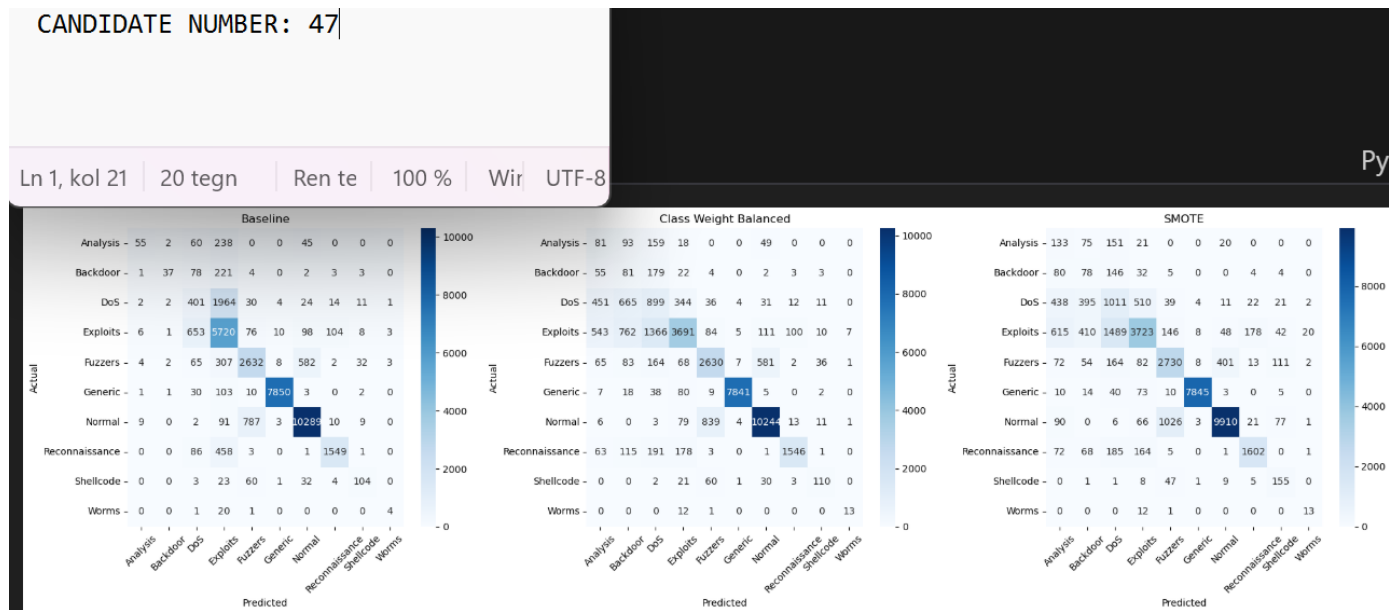
Model	Accuracy	Balanced Accuracy	Macro F1
Baseline	0.82	0.52	0.55
Class Weight	0.77	0.57	0.57
SMOTE	0.78	0.61	0.56

Accuracy means proportion of correct predictions. Balanced accuracy means average recall per class, while Macro F1 means unweighted average F1 across all classes, penalizing poor minority class performance.

This table shows that baselines accuracy is only 0.52 even though the accuracy 0.82.



This figure shows that the baseline model performs well on the generic and normal. Still it nearly ignores Worms, Backdoor and analysis.



This figure compares all three models. SMOTE improves minority class recall, but the downside is that it results with more false positives.

Personal Reflection

I still think that it was right to choose Label Encoding because 133 One-Hot columns would have been unnecessary work. One-Hot encoding could probably have been better for service and state because there were way few values (13 and 9).

Smote gave better balanced accuracy, but lower precision on analysis and backdoor. Does this mean that SMOTE is always better? Not necessarily, I think it depends on what's most important in the context.

What really surprised me was that 82% of accuracy hides that the model misses 85% of Worms. In cybersecurity this is critical because this can lead to a Worm attack that can spread around the whole network and do damage to the infrastructure.

Discussion

The strengths here are that Random Forest handles mixed feature types without much preprocessing and is robust to outliers.

The weakness is that no method solved Worms completely. 130 samples are straight up just too little.

In a real scenario is false negatives more dangerous than false positives, because you would rather flag normal traffic wrong than letting a Backdoor attack come through without knowing it.

A model that ignores minority attack classes gives the network administrators a false sense of security. This is an operational ethical risk in production systems like these.

To properly solve the Worms problem, 130 samples are simply insufficient regardless of technique. In a real-life system, the potential solutions would include more Worm traffic samples, using synthetic generation beyond SMOTE. Or treating rare attack types as anomaly detection problems rather than multi class classification.

Question 3

Introduction

Bias is when the model is too simple and can't learn the underlying patterns in the data. The results are underfitting. Variance is when the model is too complex and learns a lot of unnecessary training data. The results here are overfitting. A good model can balance these two.

Linear regression in question 1 was high bias (the learning curves were flatted out low). Random Forest on Worms in question 2 had high variance where there was a big gap between train and validation.

This question extends the analysis from question 1 and 2 by applying regularization, dimensionality reduction, and cross validation to examine how these techniques affect model generalization and stability.

Methodology

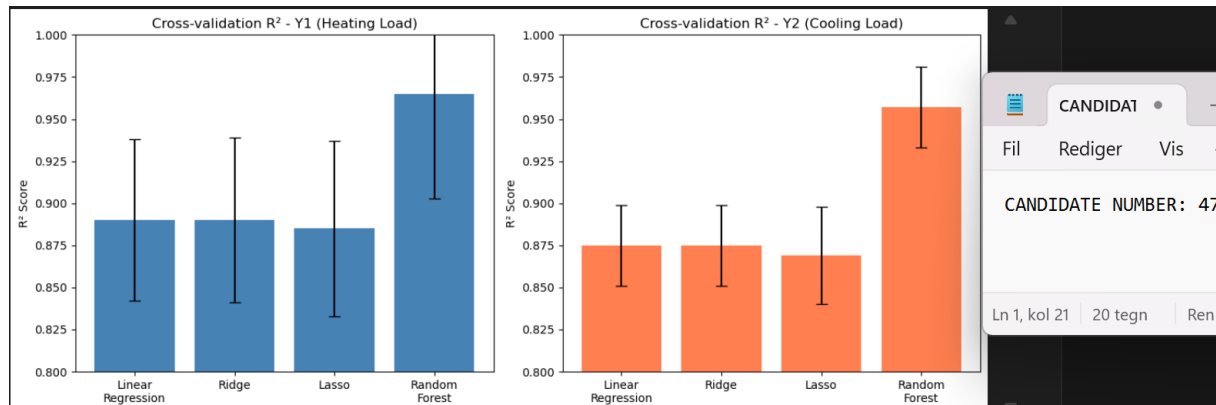
I used Regularization, PCA and Cross validation. Ridge and Lasso add a penalty to the model to reduce overfitting. Ridge shrinks all coefficients. Lasso can set some to zero.

PCA combines correlated ones into components which leads to reduction in number of features. This can be useful when multicollinearity is present.

Whereas in Cross validation the data is split 5 times and the model is evaluated on each fold instead of just one train split. This can result in a more reliable estimate of performance.

Results

For question 1:



Cross validation. Ridge and Lasso don't improve Linear Regression. Y1 and Y2 both perform almost identical to Linear Regression. This confirms high bias, and not high variance. Random Forest achieved the highest scores, but also has the largest errors on Y1, indicating some variance.

Method	Y1 R ²	Y2 R ²
Linear Regression	0.890 ± 0.048	0.875 ± 0.024
Ridge	0.890 ± 0.049	0.875 ± 0.024
Lasso	0.885 ± 0.052	0.869 ± 0.029
Random Forest	0.965 ± 0.062	0.957 ± 0.024

This table shows that Cross Validation gives a more reliable performance estimate than a single train split. The standard deviation shows that Random Forest is less stable on Y1 compared to Y2, suggesting more sensitivity to which data points end up in each fold.

PCA Components	R ²	Variance Explained
3	0.860 ± 0.127	0.77
5	0.899 ± 0.122	0.99
7	0.898 ± 0.123	1.00

5 PCA components show 99% of the variance while holding almost the same R² as the full model. This shows us that the multicollinearity observed in question 1, features like X1 and X2 carry almost

same information, which means that the dataset has fewer independent dimensions than the 8 original features suggest.

For question 2:

Method	Balanced Accuracy
Full features (34)	0.516 ± 0.022
PCA 5 Components	0.368 ± 0.021
PCA 10 Components	0.406 ± 0.023
PCA 20 Components	0.473 ± 0.023

Different from question 1, the PCA here significantly hurts performance in question 2. The balanced accuracy dropped from 0.516 to 0.368 when reduced to 5 components. This proves that network traffic features are not redundant. Each feature contributes unique information for identifying specific data types. That is why dimensionality is not a good fit for this dataset.

Personal Reflection

For question 1, the issue was high bias, regularization would not help, but Random Forest solved it by being more flexible. PCA showed that 5 components were enough and confirms the multicollinearity we found in the correlations analysis.

For question 2, variance was the partial problem for minority classes, but the real problem was class imbalance. PCA reduced the performance because the network traffic features weren't redundant like the building design features.

What really surprised me the most was that PCA had a dissimilarity effect on both question 1 and 2. In question 1 it confirmed redundancy, and in question 2 it actively hurt the performance. By this I have learned that dimensionality reduction is not universally beneficial and that it depends mostly on whether the features overlap in information.

Discussion

Cross Validation gave more stable estimates than the single split. This is especially valuable for comparing models fairly. Especially clear for Random Forest Y1 where the standard deviation was

0.062. This shows that a single split could have given misleading results. In real deployment, Cross validation is strongly recommended to use before trusting a model's performance numbers.

PCA can help when the features are correlated, like in question 1. However, it will only make things worse if each feature is unique like in question 2.

Regularization is only useful against variance and not against bias. Overall, in this question I demonstrated that no single technique universally improves model performance. The right approach depends on whether the problem suffers from bias, variance or data imbalance.

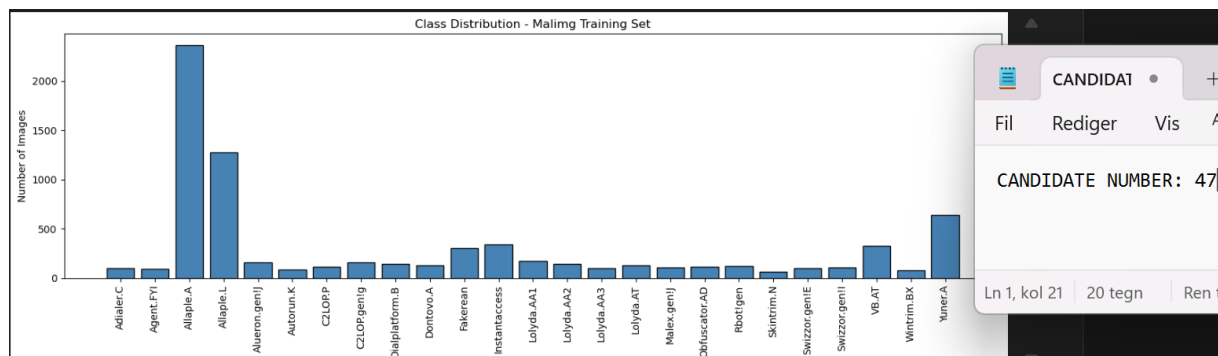
Question 4

Introduction

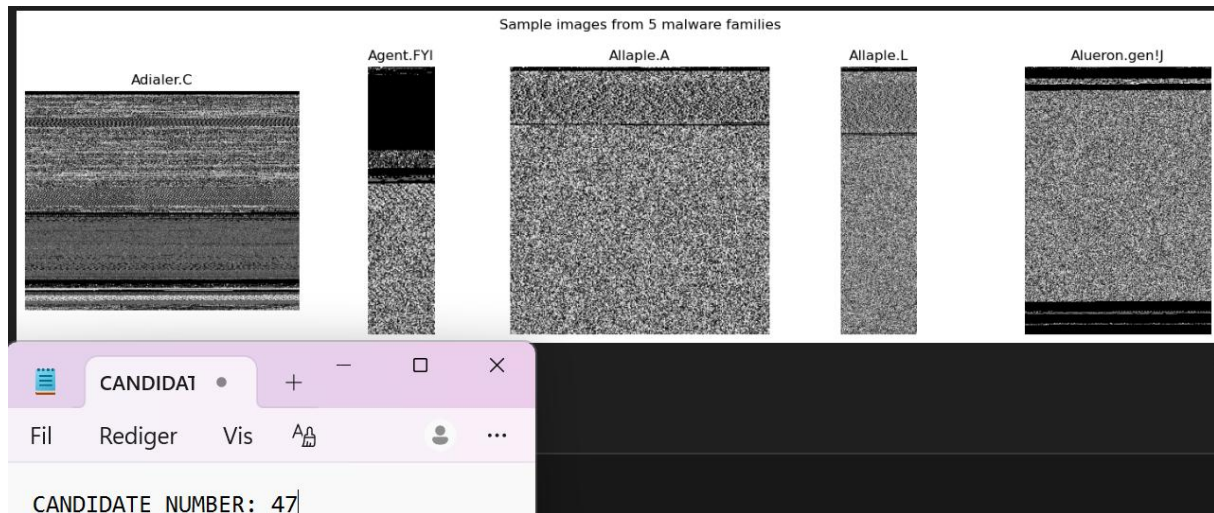
Malware classification is central in cybersecurity, making it a critical role in it. This approach uses grayscale images generated from malware binaries to classify 25 malware families (*Manmandes, n.d*) using CNN, unlike the traditional methods that rely on code analysis. Challenges in this task are class imbalance and visual similarity between families.

Dataset Understanding

The maling dataset contains 9339 images across 25 malware families split into train, val and test sets.



This chart shows significant class imbalance. The difference between the most and the least training images is 2295. Allaple.A has the most with 2359 training images, while Skintrim.N has the lowest with 64. This imbalance risks biasing the model toward majority classes.



This shows sample images from five families. Adialer.C shows horizontal patterns, while Allaple.A and Allaple.L shows that their similar with random noise patterns. This shows that these two families may be difficult to identify.

The limitation is that this dataset uses binary-to-image conversion that can lead to loss of structural information present in the original executable.

Methodology

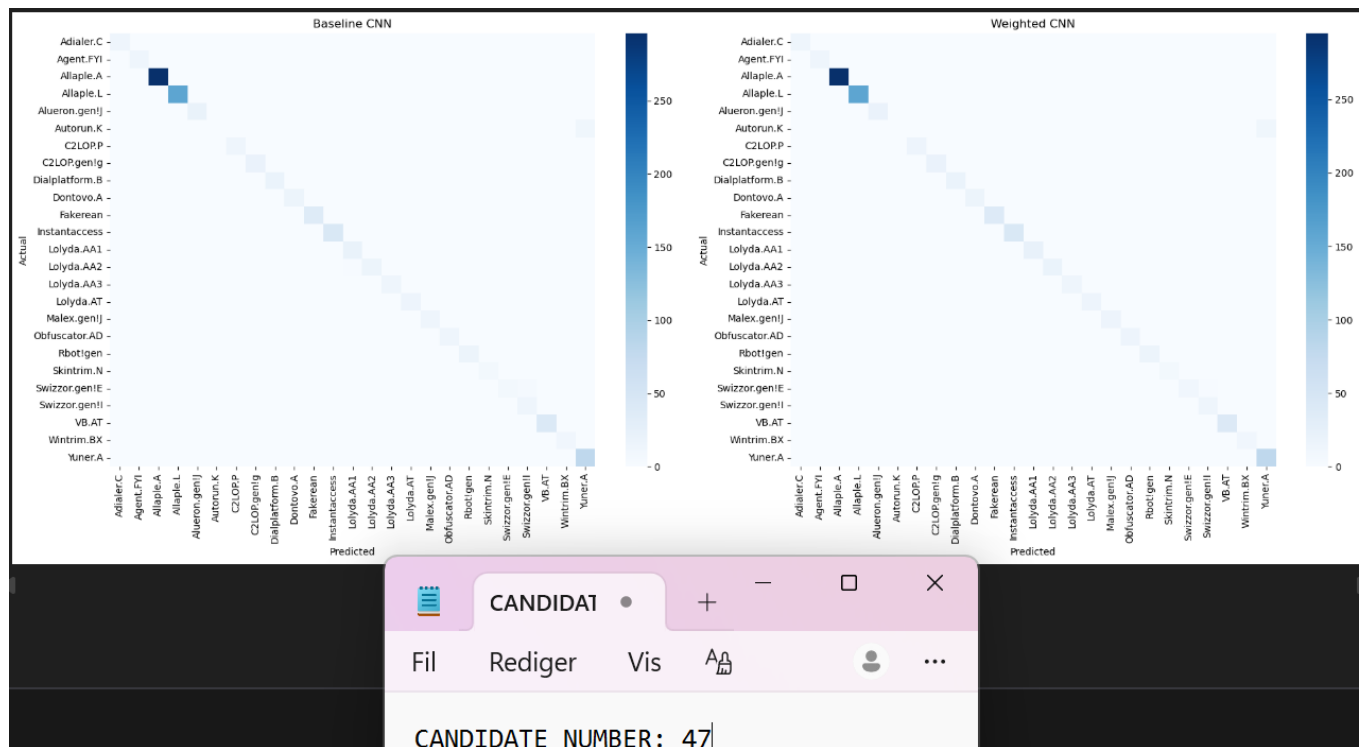
I resized all the images to 64x64 pixels and normalized them. I also designed a simple CNN with three convolutional layers followed by two fully connected layers. Applied dropout (0.5) to reduce overfitting.

Two of the models were trained. A baseline with standard CrossEntropyLoss and a weighted model where loss function was weighted inversely proportional to class frequency to address imbalance. Adam optimizer with learning rate 0.001 was used to train both models.

Results

Model	Accuracy	Balanced Accuracy	Macro F1
Baseline CNN	0.974	0.928	0.924
Weighted CNN	0.980	0.946	0.942

Weighted models barely outperform the baseline across all metrics. The improvement in balanced accuracy proves that class weights helped the model pay more attention to minority classes.



This shows that both model actually perform well overall even with strong diagonals. The most notable misclassification pattern is between Allapple.A and Allapple.L. They were consistent with their visual similarity observed in sample images. Autorun.K was not predicted by the baseline model which proves the impact of class imbalance.

Personal Reflection

I did not think that the CNN would perform that well on only 5 epochs of training on the CPU. Still, I am not fully confident that the model learns meaningful malware structure.

When I personally saw the images, Allapple.A and Allapple.L looked almost identical, still the model mostly separates them. This could mean that the CNN picks up on subtle pixel patterns, or it could maybe be memorizing training examples.

I also wonder about whether the CNN really learns meaningful malware structures, or its just exploiting superficial visual patterns like image dimensions and overall brightness. The fact that the two images are most separated despite looking identical makes me suspicious that the CNN may be picking up very low-level pixels statistics rather than structural code patterns. I think that this would be a huge problem in real deployment because a new malware variant can be designed to look visually similar to a known family and it could then fool the model entirely. If I had more time I would experiment with deeper architectures and data augmentation to verify whether the performance holds.

Discussion

The strength here is that CNN doesn't need manual feature engineering. The weakness is that visually similar families like Allapple.A and Allapple.L are hard to identify.

In real deployment the system would need retraining as new malware variants emerge. A challenge would be concept drift when new malware families emerge constantly, and a model trained on known families will fail completely on unseen variants. This proves that it is necessary to retrain fresh samples continuously.

From ethical perspective the false negatives are way more dangerous than false positives as mentioned earlier.

Sources

Question 1:

Tsanas, A. & Xifara, A. (2012). Energy Efficiency Dataset. UCI Machine Learning Repository.

<https://archive.ics.uci.edu/dataset/242/energy+efficiency>

Question 2:

Moustafa, N. & Slay, J. (2015). UNSW-NB15 dataset. Kaggle.

<https://www.kaggle.com/datasets/dhoogla/unswnb15>

Question 4:

Manmandes. (n.d). Maling Dataset. Kaggle.

<https://www.kaggle.com/datasets/manmandes/maling>